Studio Nx

백엔드 232 — plugin_service

2026-10-18

Backend(FastAPI/Python) 232 — `plugin_service` (목록 · 한 건 · 프리셋)

항목	내용
파일	`studionx/backend/app/services/plugin_service.py`
작업	S1-09 기본 악기·이펙트: 플러그인 조회 함수 넷 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	231 트랙 스키마 · 233 트랙 서비스

1. 하는 일

플러그인 정의를 읽는 함수 네 개를 둡니다. 쓰기(만들기·고치기)는 없습니다. 플러그인은 시드 SQL(242)로 넣고 화면은 읽기만 하기 때문입니다.

함수	하는 일	권한
`list_plugins(db, *, active_only=True)`	종류, 이름 순으로 플러그인 목록. 기본은 쓸 수 있는 것만	로그인한 회원이면 누구나
`get_plugin(db, plugin_id)`	한 건. 없으면 None	위와 같음
`require_plugin(db, plugin_id)`	한 건. 없으면 404 오류를 냅니다	위와 같음
`list_plugin_presets(db, plugin_id, user_id=None)`	시스템 프리셋과 내 프리셋	위와 같음

`user_id` 를 주면 `owner_id IS NULL`(시스템) 이거나 `owner_id = user_id`(내 것) 인 행만 골라 이름 순으로 돌려줍니다. 주지 않으면 시스템 프리셋만 돌려줍니다.

2. 언제 쓰나

쓰는 곳	
`GET /plugins` · `GET /plugins/{id}` · `GET /plugins/{id}/presets` (237)	라우터가 이 함수들을 부릅니다
`track_service.add_instrument` · `add_effect` (234·235)	`require_plugin` 으로 플러그인이 있는지 확인하고 `plugin_type` 을 검사합니다
화면의 악기/이펙트 고르기 목록	`list_plugins` 결과를 그대로 그립니다

3. 코드

목록과 한 건

```
def list_plugins(db: Session, *, active_only: bool = True) -> Sequence[Plugin]:
    stmt = select(Plugin).order_by(Plugin.plugin_type, Plugin.name)
    if active_only:
        stmt = stmt.where(Plugin.is_active == True)  # noqa: E712
```

```
    return db.execute(stmt).scalars().all()


def get_plugin(db: Session, plugin_id: int) -> Plugin | None:
    return db.get(Plugin, plugin_id)


def require_plugin(db: Session, plugin_id: int) -> Plugin:
    p = get_plugin(db, plugin_id)
    if p is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Plugin not found.")
    return p
```

프리셋

```
def list_plugin_presets(
    db: Session, plugin_id: int, user_id: int | None = None
) -> Sequence[PluginPreset]:
    stmt = select(PluginPreset).where(PluginPreset.plugin_id == plugin_id)
    if user_id is not None:
        stmt = stmt.where(or_(PluginPreset.owner_id.is_(None), PluginPreset.owner_id == user_id))
    else:
        stmt = stmt.where(PluginPreset.owner_id.is_(None))
    stmt = stmt.order_by(PluginPreset.name)
    return db.execute(stmt).scalars().all()
```

문법 풀이

식	뜻
`def list_plugins(db, *, active_only=True)`	`*` 뒤 인자는 이름을 붙여야만 넘길 수 있습니다. `list_plugins(db, False)` 같은 실수를 막습니다
`Plugin.is_active == True`	SQL 로 바뀌면 `is_active = 1` 입니다. 파이썬 문법 검사(ruff E712)가 걸려 `noqa` 를 달았습니다
`db.get(Plugin, plugin_id)`	기본 키로 한 건을 찾습니다. 없으면 None 을 돌려줍니다
`raise AppError(ErrorCode.NOT_FOUND, 404, "...")`	오류 코드·상태·메시지 세 가지를 함께 넘겨야 합니다. 화면은 `{"code": "NOT_FOUND"}` 를 받습니다
`or_(A, B)`	SQL 의 `OR` 입니다. 시스템 프리셋과 내 프리셋을 한 번에 고릅니다
`is_(None)`	SQL 의 `IS NULL` 입니다. `== None` 대신 써야 의도가 분명합니다

4. 왜 이렇게

선택	이유
읽기만 두 개	플러그인 정의는 시드로만 바뀝니다. API 로 만들면 화면 구현이 없는 플러그인이 표에 들어갈 수 있습니다
`require_plugin` 을 따로	"없으면 404" 를 여러 곳(237, 234, 235)에서 씁니다. 한 곳에 모아야 메시지와 코드가 어긋나지 않습니다
`user_id` 를 키워드로	라우터가 `current_user.user_id` 를 넘깁니다. 값이 없으면 시스템 프리셋만 나옵니다
프리셋을 이름 순으로	화면의 선택 목록에서 찾기 쉽습니다. `Init` 이 맨 앞에 옵니다

선택	이유
정렬을 `plugin_type` → `name`	화면 목록이 악기부터 보이고, 같은 종류 안에서는 이름 순입니다

5. 시험

`tests/test_plugin_routes.py` 가 다음을 확인합니다.

- 목록 응답의 칸 이름과 `active_only=false` 일 때 꺼진 플러그인도 나오는지
- 상세 응답의 `param_schema` 항목과 없는 번호일 때 404
- 프리셋 목록에 시스템 프리셋과 내가 저장한 프리셋만 나오고 남의 것은 빠지는지
- 로그인하지 않으면 401